

백엔드 개발

2026년 여름 비교과과정 바이브코딩 2

Superpowers

- 사양 중심 개발을 도와주는 도구

"사양 중심 개발(Spec-Driven Development)은 전통적인 소프트웨어 개발 방식을 완전히 뒤바꿉니다. 지난 수십 년 동안 '코드'는 왕과 다름없었습니다. 상세 사양서(Specification)는 그저 '진짜 작업'인 코딩이 시작되기 전까지만 쓰이다 버려지는 임시 가설물에 불과했죠.

하지만 사양 중심 개발은 이 구도를 바꿉니다. 이제 사양서는 단순히 코딩을 안내하는 역할에 그치지 않고, **직접 실행 가능한 형태**가 되어 작동하는 구현체를 즉각적으로 생성해 냅니다."

- 테스트 주도 개발 (TDD)

- 서브에이전트 방식

Superpowers의 다양한 명령/스킬

- :brainstorming
- :writing-plans
- :executing-plans
- :test-driven-development
- :using-git-worktrees
- :systematic-debugging

CLAUDE.md

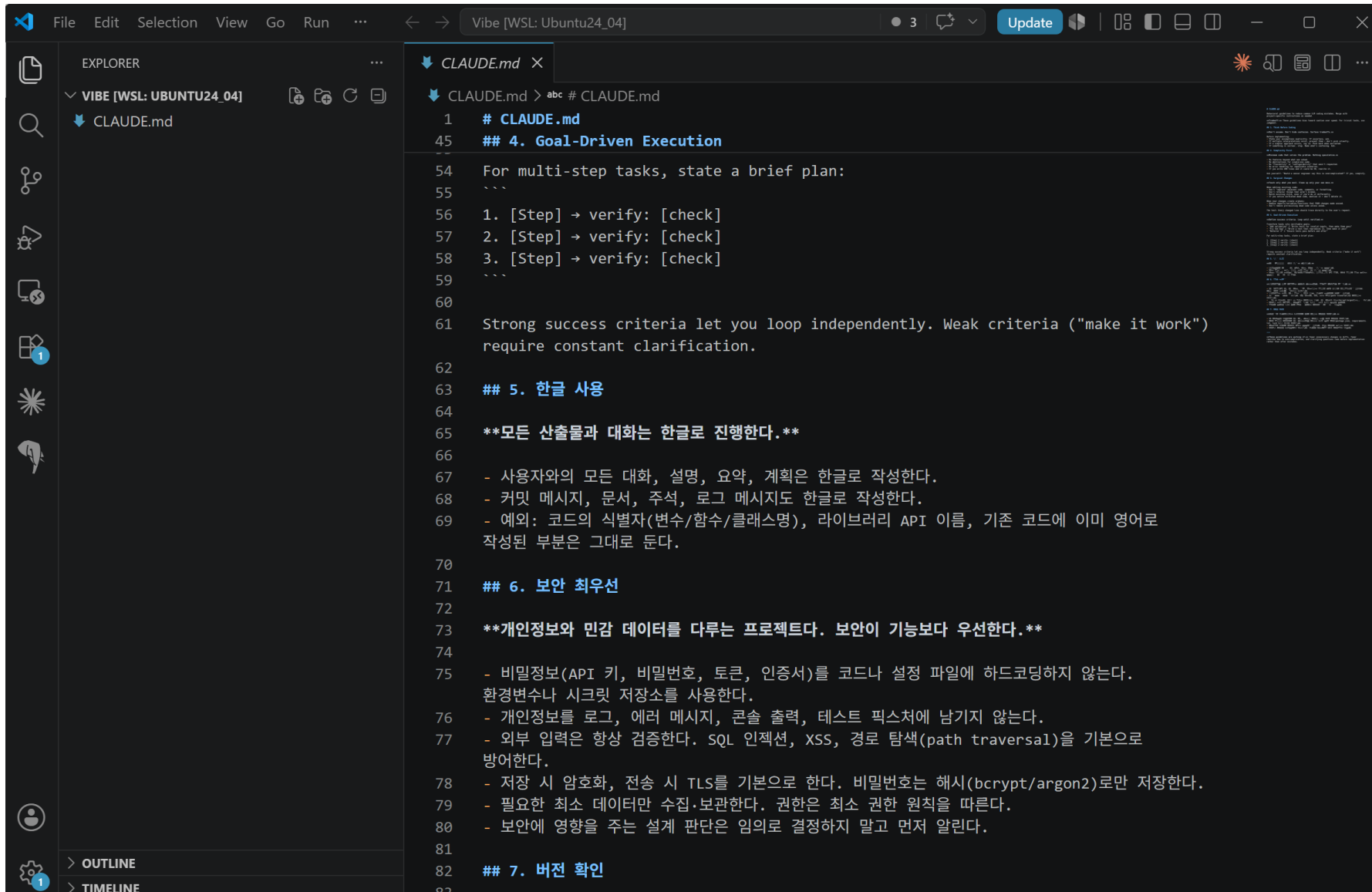
- 프로젝트 원칙 설정
- 목적: 프로젝트의 '헌법'을 정의합니다. 기술 스택, 코딩 컨벤션, 아키텍처 원칙 등 변하지 않는 규칙을 설정합니다.
- 예) 안드레이 카파시의 제안

```
curl https://raw.githubusercontent.com/forrestchang/andrej-karpathy-skills/main/CLAUDE.md >> CLAUDE.md
```

- 예) 직접 추가하기

› 다음의 내용으로 CLAUDE.md 파일에 추가할 것. 모든 산출물과 대화는 한글로 진행되어야 함. 사용자의 개인정보 및 민감한 데이터를 다루는 만큼, 보안을 최우선으로 고려해야 함. 특정 패키지나 소프트웨어를 사용하려면 반드시 버전을 확인할 것

manual mode on



/superpowers:brainstorming

- 프로젝트의 첫 시작포인트
- 목적: 구현하고자 하는 기능의 'What(무엇을)'과 'Why(왜)'에 집중하여 상세 요구사항을 작성합니다. (기술적인 내용이 아니라, 사용자 입장에서)

› `/superpowers:brainstorming` 이 서비스는 취업정보 사이트입니다. 사용자가 웹브라우저로 접근하여 현재 채용이 진행 중인 회사의 정보를 확인할 수 있어야 합니다. 제공되는 정보로는 회사명, 회사위치, 채용분야, 채용조건, 지원기간, 지원방법, 예상급여 등입니다. 정보를 만드는 것은 관리자입니다. 관리자는 채용 정보가 들어 있는 pdf 파일을 업로드할 수 있고, 채용 정보가 있는 url을 등록할 수도 있습니다. 관리자는 시스템에 저장된 정보를 수정/삭제할 수 있어야 합니다. 일반 이용자들은 로그인 후 정보를 확인할 수 있습니다.■

- 프로젝트 디렉터리는 **CLAUDE.md** 하나뿐인 빈 상태입니다 (**git** 저장소도 아직 아님). 새로 시작하는 프로젝트로 보고 진행하겠습니다.

브레인스토밍 스킬로 설계를 먼저 잡겠습니다. 체크리스트:

1. 프로젝트 맥락 파악 ☒ (빈 디렉터리, 신규 프로젝트)
2. 질문으로 요구사항 구체화 ← 지금 여기
3. 접근 방식 2~3개 비교 제안
4. 설계안 제시 및 승인
5. 스펙 문서 작성 (**docs/superpowers/specs/**)
6. 스펙 자체 검토 → 사용자 검토
7. **writing-plans** 스킬로 전환

말씀하신 내용에서 가장 불확실하고 프로젝트 규모를 좌우하는 지점이 하나 보입니다: **PDF** 파일과 **URL**은 자유 형식인데, 화면에는 회사명·위치·채용분야·조건·기간·방법·급여라는 정형 필드로 보여줘야 합니다. 이 변환을 누가/어떻게 하느냐에 따라 만들 것이 완전히 달라집니다.

다만 그걸 묻기 전에 판단 기준이 되는 것부터 확인하겠습니다.

□ 프로젝트 성격

이 서비스의 목적과 운영 범위는 어느 쪽에 가깝습니까?

- 1. 수업/실습용 데모
강의나 학습 목적. 실제 사용자 트래픽 없음. 로컬 또는 간단한 배포. 완성도보다 동작하는 전체 흐름이 중요.
- 2. 학과 내 실제 운영
학생들이 실제로 보는 취업정보 사이트. 관리자 1~2명, 이용자 수십~수백명 규모. 안정성과 데이터 정확성 필요.
- 3. 공개 서비스 지향
불특정 다수 대상. 트래픽 확장, 보안, 운영 자동화까지 고려한 본격 서비스.

문서 정보 추출방법

- PDF 파일이나 웹페이지에 있는 내용에서 원하는 정보를 추출하려면 '문서 구조 분석'이 필요하다.
- 과거에는 규칙에 기반하여 추출 코드를 작성했다 -> 파싱
- 현재는 LLM을 이용하여 정보를 추출한다.
 - LLM 서비스에서 제공하는 API 사용
 - 이를 위해서는 일반적으로 API Key를 발급받아야 함
 - 클로드, 코덱스, 안티그래비티 모두 API Key를 제공하지만 별도의 과금을 할 수 있다
 - 실습에서는 자체 LLM(Gemma 4)의 API를 사용한다.

LLM 설정값

- 학과에 설치된 서버를 통해 OpenAI API 호환 서비스 제공
 - 서버 주소 (swagger-ui)
 - <http://203.255.224.60:4000>
 - LLM API endpoint
 - <http://203.255.224.60:4000/v1>
 - OpenAI API (chat completion)
 - <http://203.255.224.60:4000/v1/chat/completions>
 - 주의사항
 - 반드시 temperature 값을 0으로 지정해줄 것.
- ✓ 할당된 key를 사용할 것 **!!! 절대 GitHub에 올리면 안 됨 !!!**

기술 스택 지정

수업/실습용 데모라는 전제에서, 어떤 기술 스택으로 만들까요?

› 1. **Python + FastAPI + SQLite (추천)**

PDF 텍스트 추출·웹 본문 파싱 라이브러리가 가장 성숙하고, Jinja2 서버 렌더링이라 별도 빌드 없이 프로세스 하나로 실행됩니다. 총 코드량이 가장 적고 수업에서 흐름을 따라가기 쉽습니다.

2. **Node.js + Express + SQLite**

JavaScript 하나로 서버와 브라우저를 모두 다룹니다. 수업이 자바스크립트 중심이라면 적합. PDF 텍스트 추출은 pdf-parse 등으로 가능하나 Python보다 선택지가 적습니다.

3. **Next.js 풀스택**

React 기반으로 화면을 현대적으로 만들기 좋고 배포도 간편합니다. 대신 프론트/서버 경계, 빌드 도구, 의존성이 늘어 데모 기준으로 구조가 무거워집니다.

4. **Django**

관리자 페이지(admin)가 기본 제공되어 수정/삭제 CRUD를 거의 공짜로 얻습니다. 대신 프레임워크 고유 관습(앱 구조, ORM, 설정)이 많아 설명할 것이 늘어납니다.

5. **Type something.**

6. Chat about this

Enter to select · ↑/↓ to navigate · Esc to cancel

› Spring Boot + React 로 하겠습니다

⏏ manual mode on

채용정보를 저장할 데이터베이스는 무엇으로 할까요?

› 1. H2 파일 모드 (추천)

별도 설치·서버 실행 없이 Spring Boot가 직접 파일로 관리합니다. 서버를 꺼도 데이터가 남고, 실습환경 재현이 쉬울겁니다. 나중에 PostgreSQL로 옮기도 쉬움.

2. PostgreSQL (Docker)

실무에서 가장 많이 쓰는 구성. Docker로 컨테이너를 띄워야 하므로 실행 절차가 한 단계 늘어나지만, 실무에 가까운 경험을 줍니다.

3. MySQL (Docker)

국내 실무/교육 현장에서 친숙도가 높은 DB. PostgreSQL과 사용감은 비슷하며 역시 Docker 실행이 필요합니다.

4. Type something.

5. Chat about this

Enter to select · ↑/↓ to navigate · Esc to cancel

관리자 로그인은 어떤 방식으로 할까요? (일반 이용자는 로그인 없이 조회 - 이는 확정)

› 1. 세션 쿠키 로그인 (추천)

Spring Security 기본 방식. 로그인 성공 시 HttpOnly 쿠키를 내려주고 서버가 세션을 관리합니다. 토큰을 브라우저에 저장할 필요가 없어 XSS 탈취 위험이 낮고, 로그아웃·만료 처리가 단순합니다.

2. JWT 토큰

로그인 시 토큰을 발급받아 React가 요청마다 헤더에 실어 보내는 방식. SPA에서 흔히 쓰이고 모바일 앱 확장에 유리하지만, 토큰 저장소·만료·갱신을 직접 다뤄야 해 코드가 늘고 실수 여지가 큼.

3. 관리자 계정도 DB로 관리

위 둘 중 어떤 방식이든, 관리자를 여러 명 등록·추가할 수 있게 DB 테이블로 관리합니다. 계정 관리 화면과 CRUD가 추가로 필요해 데모 범위가 커집니다.

4. Type something.

5. Chat about this

Enter to select · ↑/↓ to navigate · Esc to cancel

- 주요 결정이 모두 정해졌습니다. 이제 전체 구성 방식을 놓고 세 가지 안을 비교하겠습니다. 세 안 모두 지금까지 정한 것(Spring Boot + React, H2 파일, 세션 로그인, AI 추출 후 검수, URL 1회 수집, 검색·필터)은 동일하고, 차이는 "프론트와 백엔드를 어떻게 묶느냐"와 "AI 추출을 동기로 처리하느냐" 두 가지뿐입니다.

A안 - 단일 프로젝트, 동기 추출 (추천)

하나의 Spring Boot 프로젝트 안에 `frontend/` 디렉터리로 Vite+React를 둡니다. 개발할 때는 Vite 개발 서버(5173)가 API 요청을 8080으로 프록시하고, 배포할 때는 React 빌드 결과를 Spring의 정적 리소스로 넣어 JAR 하나로 실행합니다. AI 추출은 관리자가 업로드 버튼을 누르면 그 자리에서 호출하고, 몇 초 뒤 채워진 검수 폼이 뜹니다.

→ 실행 명령이 하나, 배포도 하나. 수업에서 "이거 받아서 `java -jar`만 하면 됩니다"가 가능합니다. 대신 추출 중 수 초간 로딩 스피너를 봐야 합니다.

B안 - 프론트/백 완전 분리

React와 Spring Boot를 별개 프로젝트로 두고 항상 두 프로세스로 띄웁니다. CORS와 쿠키 설정을 명시적으로 다뤄야 합니다. → 실무의 분리 배포 구조를 그대로 보여주지만, 데모에서 얻는 것 대비 설정 부담(CORS, credentials, 두 개의 실행 절차)이 큼니다.

C안 - 비동기 잡 기반 추출

업로드하면 즉시 "처리중" 상태로 응답하고, 백그라운드에서 추출한 뒤 화면이 상태를 폴링합니다.

→ 큰 PDF나 느린 사이트에도 화면이 멎지 않지만, 잡 상태 테이블·폴링·실패 재시도가 추가로 필요합니다. 관리자 한 명이 쓰는 데모에는 과합니다.

› B안으로 진행합니다. 그리고 Claude API가 아닌 OpenAI API로 변경바랍니다.■

섹션 2 - 데이터 모델

핵심 엔티티는 **JobPosting** 하나입니다. 데모 규모에서 테이블을 더 쪼갤 이유가 없습니다.

여기서 중요한 설계 판단이 하나 있습니다. 검색·필터를 하려면 자유 텍스트로는 안 됩니다. 공고마다 "서울시 강남구 테헤란로...", "경기 성남 판교"처럼 제각각으로 적혀 있으면 "서울" 필터가 동작하지 않습니다. 그래서 필터용 정규화 필드와 원문 표시용 필드를 나눠서 둡니다.

필드	용도	설명
companyName	표시·검색	회사명. 유일한 필수 필드
region	필터	시/도 단위로 정규화 (서울, 경기, 부산...). 못 찾으면 비움
locationText	표시	공고에 적힌 주소 원문
jobCategory	필터	미리 정한 분류 중 하나 (개발, 디자인, 영업, 생산, 사무 등)
jobFieldText	표시	"백엔드 개발자(신입)"처럼 공고 원문 그대로

PDF 텍스트 추출은 작은 샘플 PDF를 픽스처로 씁니다. 픽스처에는 실제 회사·담당자 정보를 쓰지 않고 가상의 데이터를 넣습니다.

프론트엔드 테스트는 데모 규모를 감안해 목록·검수 폼 렌더링 수준으로 최소화합니다. 백엔드 로직 검증에 무게를 둡니다.

이것으로 설계가 마무리됩니다. 이 섹션이 확인되면 전체를 스펙 문서로 정리해 [docs/superpowers/specs/2026-07-25-job-board-design.md](#)에 쓰고 커밋하겠습니다. (현재 디렉터리가 git 저장소가 아니라 `git init`부터 하게 되는데, 진행해도 될까요?)

- 설계 문서를 작성해 커밋했습니다: [docs/superpowers/specs/2026-07-25-job-board-design.md](#)

자체 검토에서 모호했던 세 곳을 채웠습니다:

- 검색 대상 - `keyword`는 회사명과 분야명 원문만 검색 (원문 전체는 제외)
- 정렬 규칙 - 마감 임박순, 상시채용(마감일 없음)은 뒤로 보내고 최근 등록순
- 관리 목록 - 페이지네이션 없이 전체 반환

스펙을 검토해 주시고, 수정할 부분이 있으면 말씀해 주세요. 특히 확인해 보실 만한 지점:

- 4장 데이터 모델 - 필드 구성이 학과에서 실제로 보여주고 싶은 정보와 맞는지
- 6장 API의 게시 조건 - 회사명만 필수로 뒀는데, 마감일이나 지원방법도 필수로 해야 할지
- 1장 제외 범위 - 마감된 공고 조회 화면을 뺐는데 지난 채용 이력을 참고하고 싶으실 수도 있습니다

승인해 주시면 `writing-plans` 스킴으로 구현 계획을 작성하겠습니다.

휴식

/superpowers:writing-plans

- 구현 계획

- 목적: 'How(어떻게)'를 결정합니다. 사양서를 바탕으로 데이터 모델, API 설계, 컴포넌트 구조 등 기술적 이행 계획을 수립합니다.

- Skill(superpowers:writing-plans)
 └ Successfully loaded skill
- writing-plans 스킬로 구현 계획을 작성하겠습니다. 그 전에 계획에 코드를 적으려면 확인해야 할 API가 두 가지 있습니다.
- Fetch(<https://raw.githubusercontent.com/openai/openai-java/main/README.md>)

실행 방식을 선택해 주세요.

1. 서버에이전트 방식 (권장) - 태스크마다 새 에이전트를 띄우고 사이사이 제가 검토합니다. 컨텍스트가 깨끗하게 유지되어 후반 태스크의 정확도가 높습니다.
2. 인라인 실행 - 이 세션에서 순서대로 진행하고 중간중간 확인 지점을 둡니다. 진행 상황이 한눈에 보이지만 대화가 길어질수록 초반 태스크의 세부사항을 놓칠 수 있습니다.

어느 쪽으로 할까요?

/superpowers:subagent-driven-development

- 구현
- 목적: 생성한 plan에 맞춰 코드 생성. 서브에이전트 방식으로 진행

```
● Skill(superpowers:subagent-driven-development)
  └─ Successfully loaded skill
```

- 서브에이전트 방식으로 실행하겠습니다. 먼저 격리된 작업 브랜치와 진행 원장(ledger)을 준비합니다.
- Running 2 shell commands...
 - └─ \$ bash /home/jblee/.claude/plugins/cache/claude-plugins-official/superpowers/6.2.0/skills/subagent-driven-development/scripts/sdd-workspace /home/jblee/Courses/Vibe/docs/superpowers/plans/2026-07-25-job-board.md

약 4시간 후

- SDD 실행이 끝났습니다. 최종 리뷰까지 클린이지만, 계획서가 요구한 수동 확인 1건이 남아 있어 워크스페이스를 아직 지우지 않았습니다.

재개 후 진행한 것

단 계	결 과
Task 14 리뷰 (중단 지점)	spec  / Approved, 수정 없음
Task 15 검수 품	구현 → Important 1건(plan-mandated) → 사용자 판정 → fix 1라운드 → 클린
Task 16 관리 목록 + README	구현 → Important 1건(plan-mandated) → 사용자 판정 → fix 1라운드 → 클린
최종 전체 브랜치 리뷰 (30커밋, opus)	Critical 1 + Important 5
최종 fix wave 1회 + 범위 재리뷰	6건 전부 ADDRESSED, 새 Critical/Important 없음

최종 리뷰가 잡아낸 것

최종 리뷰어가 자동 테스트로는 구조적으로 검증 불가능한 7항목을 추가로 지목했습니다 - 그중 두 가지는 특히 중요합니다:
 - 11MB PDF 업로드 → MockMvc가 서블릿 multipart 한도를 우회해서 이걸 실서버에서만 확인됩니다 (그래서 위양성 테스트를 커밋 [26adac8](#)에서 삭제했습니다)
 - 로그인 직후 바로 PDF 업로드 → I2 수정이 프론트까지 제대로 반영됐는지 확인하는 지점. 403이 나면 안 됩니다

전체 목록은 원장([.superpowers/sdd/2026-07-25-job-board/progress.md](#)) 마지막에 기록해 뒀습니다.

수동 확인을 진행하시겠습니까? 백엔드·프론트를 띄우는 것까지는 제가 도울 수 있습니다. 확인이 끝나면 [superpowers:finishing-a-development-branch](#)로 병합 옵션을 정리하고 워크스페이스를 정리하겠습니다.

/superpowers:verification-before-completion

- 완료하기 전에 확인하기
- 목적: 생성된 결과물이 계획대로 만들어졌는지 확인하기

› /superpowers:verification-before-completion

- 검증 없이 테스트 건수를 주장했습니다. 서브에이전트 리포트와 재리뷰어 확인에만 근거했고, 이 세션에서 직접 실행하지 않았습니다. 지금 확인합니다.

자동 검증 가능한 항목은 전부 통과했습니다. 마지막 기준 하나가 브라우저 수동 확인이라 아직 미충족이고, 이것 때문에 워크스페이스를 지우지 않고 원장에 남겨 뒀습니다.

수동 확인을 진행하시겠습니까? 백엔드·프론트를 띄우는 것까지 제가 하고, 확인 항목을 순서대로 안내하겠습니다.

* Baked for 2m 20s

> ok

- 수동 확인을 진행하겠습니다. 먼저 실행 환경을 점검합니다.

Ran 2 shell commands

- 포트는 비어 있습니다. 시작 전 세 가지 결정이 필요합니다.

한 가지 미리 알려드릴 점: 기존 개발 DB([backend/data/jobboard.mv.db](#))에 게시된 공고가 남아 있습니다. Step 8의 1번 항목이 *" / 접속 → 진행중인 채용공고가 없습니다. 표시"*를 기대하므로, 기존 데이터가 있으면 이 항목을 그대로 확인할 수 없습니다.

- User declined to answer questions

- └ · 관리자 비밀번호 해시를 어떻게 준비할까요? (직접 생성하기 (권장) / 제가 임시번호로 생성)
- OPENAI_API_KEY를 사용할 수 있나요? (Step 8의 9번 PDF 자동추출 항목에 필요) (있음 - 직접 설정 / 없음 - 건너뛰기)
- 기존 개발 DB를 어떻게 할까요? (백업 후 초기화 (권장) / 그대로 두기)

- 어떤 점을 확인하고 싶으신가요?

첫 번째 질문에는 "직접 생성"을 고르셨고, 나머지 두 개 (OpenAI 키, 기존 개발 DB)는 답변이 없었습니다. 이 중 궁금한 부분이 있으시면 말씀해 주세요 - 아니면 다른 부분이 불명확했는지 알려주시면 그에 맞춰 다시 여쭙겠습니다.

* Worked for 1m 26s

> OpenAI 키를 사용하기에 앞서서 OpenAI API를 위한 url을 <http://llm.ycbjkim.com> 으로 변경해야 합니다. OpenAI 키 값은 `sk-CuKcOn2BV_nIbXuNjtKtKg` 입니다.

수동 테스트가 필요할 수도...

남은 것 - 브라우저에서 봐 주셔야 합니다

DB를 비워 뒀으니 <http://localhost:5173> 에서 1번부터 순서대로 진행하시면 됩니다 (admin / 설정하신 비밀번호):

1. / → "진행중인 채용공고가 없습니다." 문구
2. /admin/jobs → 로그인 화면으로 리다이렉트 (RequireAuth)
3. 로그인 → 관리 목록 진입
4. 새 공고 등록 → 직접 입력 → 빈 양식 → 검수 폼 오른쪽에 원문 영역 표시
5. 회사명 비우고 게시 → 화면에 에러 메시지 표시
6. 채우고 게시 → 목록에 "게시됨"
7. / → 공고 보임 → 클릭 → 상세
8. 삭제 → 확인 대화상자 → 취소하면 안 지워지고, 확인하면 지워짐

시간·환경 제약으로 남긴 것: 세션 30분 만료 후 동작, ADMIN_PASSWORD_HASH 미설정 기동, 실제 채용 사이트 URL의 본문 추출 품질(nav/header/footer 일괄 제거가 본문을 지우는지 - 확인하실 URL을 주시면 제가 돌려보겠습니다).

서버를 내릴 때 말씀해 주세요. 백업한 DB(jobboard.mv.db.backup-20260726-155135)를 복원할지도 그때 정하시면 됩니다.

* Sautéed for 15m 49s

※ recap: 채용정보 게시판 SDD 실행을 마쳤고, 마지막 남은 수동 검증 중입니다. API 레벨 확인은 전부 통과했으니, 이제 브라우저에서 <http://localhost:5173> 에 접속해 Step 8의 화면 항목(로그인 리다이렉트, 검수 폼, 삭제 확인 대화상자)을 봐 주시면 됩니다. (disable recaps in /config)

- 서버를 정리하고 브랜치 마무리 단계로 넘어가겠습니다.

Ran 3 shell commands

- Background command "Start backend server" failed with exit code 1
- 백엔드는 내려갔고 vite만 남았습니다 (앞선 `pkill -f vite`가 그 문자열을 담은 제 셸 자신을 잡아서 실패했습니다). PID로 종료합니다.

Ran 1 shell command

- Background command "Start frontend dev server" failed with exit code 143
- 서버 정리 완료, 워킹 트리 클린입니다. 브랜치 마무리 스킬을 불러옵니다.
- Skill(superpowers:finishing-a-development-branch)
 - └ Successfully loaded skill
- finishing-a-development-branch 스킬로 작업을 마무리하겠습니다.

먼저 통합 대상 트리에서 전체 테스트를 다시 돌립니다.

Ran 3 shell commands

- 테스트 통과: 백엔드 78건, 프론트엔드 29건, 둘 다 exit 0.

환경 확인: 일반 저장소(워킹트리 아님). `feature/job-board`는 `master(090a1ea)`에서 분기했습니다 - merge-base로 확인했습니다. 원격이 설정되어 있지 않아 PR 옵션은 원격 추가가 선행돼야 합니다.

Implementation complete. What would you like to do?

1. Merge back to master locally
2. Push and create a Pull Request
3. Keep the branch as-is (I'll handle it later)

Swagger-UI 지원 추가

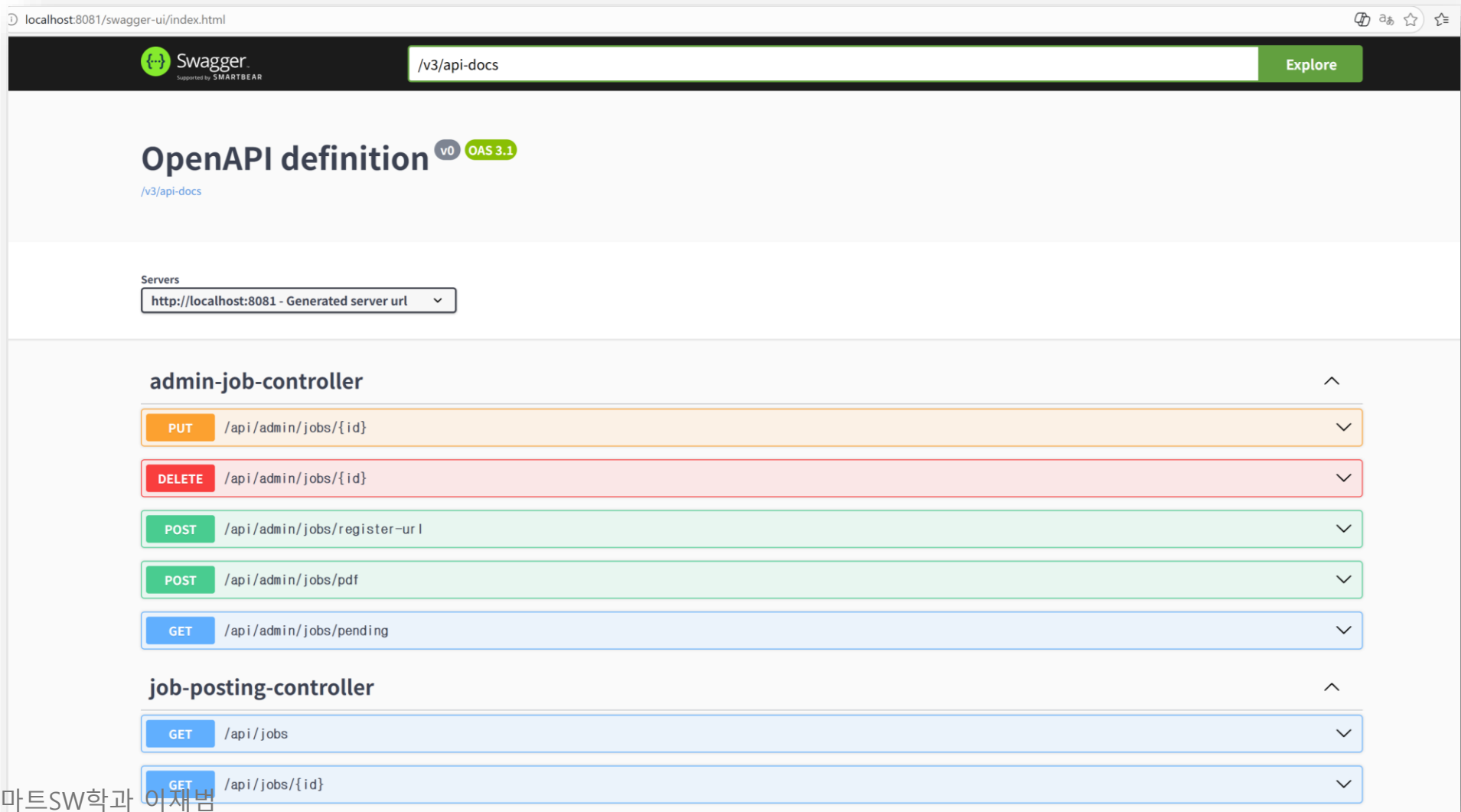
› Swagger UI 페이지를 지원하도록 수정바랍니다.

- Skill(superpowers:brainstorming)
└ Successfully loaded skill

- 프로젝트 상태를 먼저 파악하겠습니다.

Searched for 1 pattern, read 2 files, listed 1 directory, ran 2 shell commands

확인 (localhost:8080/swagger-ui.html)



확인 (localhost:8080/h2-console)

English [Preferences](#) [Tools](#) [Help](#)

Login

Saved Settings:

Setting Name:

Driver Class:

[JDBC URL:](#)

User Name:

Password: